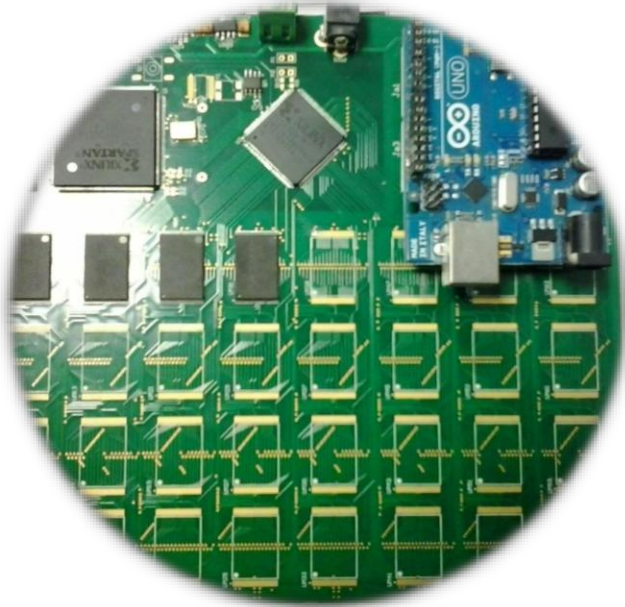


SISTEMAS EMBEBIDOS



CONTENIDO

- **Introducción**
- Revisión de Teoría básica de Microprocesadores
- Procesador embebido ARM

HISTORIA

Los microprocesadores y los microcontroladores comparten una historia inicial común hasta mediados de la década de los 70's.

Company	Processor	Year
INTEL 4004	4-bit	1971
INTEL 8085	8-bit	1974
INTEL 8048	8-bit	1976
INTEL 8031	8-bit(ROM-LESS)	-
INTEL 8051	8 bit(MASK ROM)	1980
INTEL 8086	16-bit	1978
Atmel At89C51	8-bit(Flash Memory)	1984
Microchip PIC16C64	8-bit	1985
Motorola 68HC11	8-bit(on chip ADC)	1985
AVR	8-bit RISC	1996

CARACTERÍSTICAS BÁSICAS

DEFINICIÓN DE MICROCONTROLADOR – SISTEMA EMBEBIDO

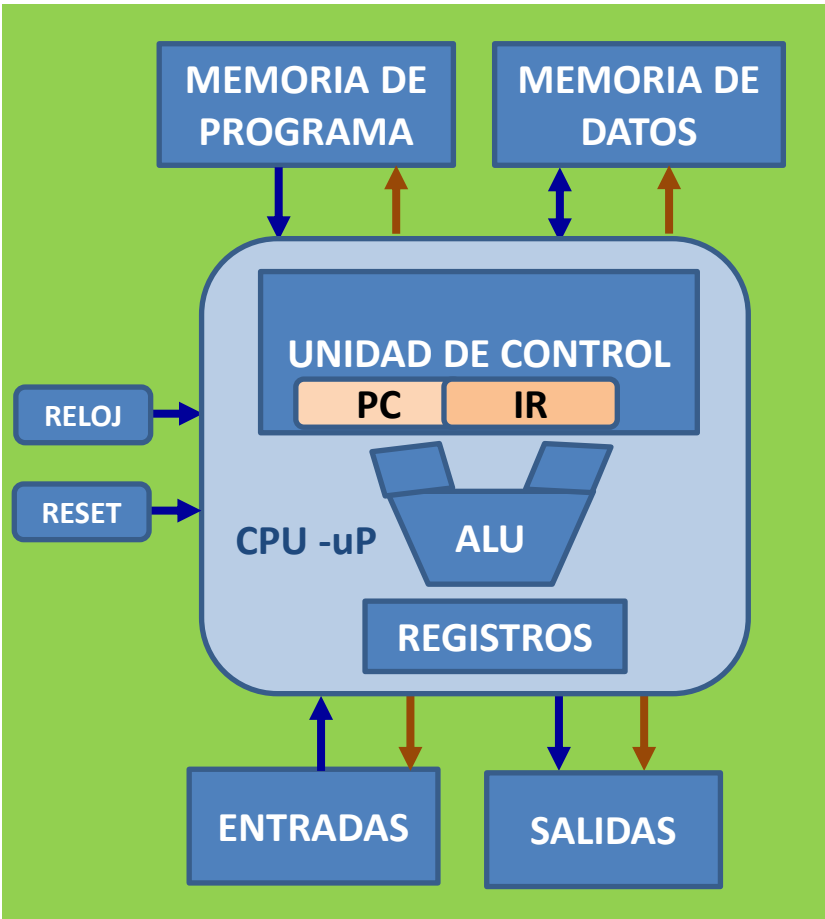
- Es un circuito integrado programable el cual contiene todos los componentes de un computador.
- Es un computador completo de limitadas prestaciones, que está contenido en un único chip.

COMPONENTES

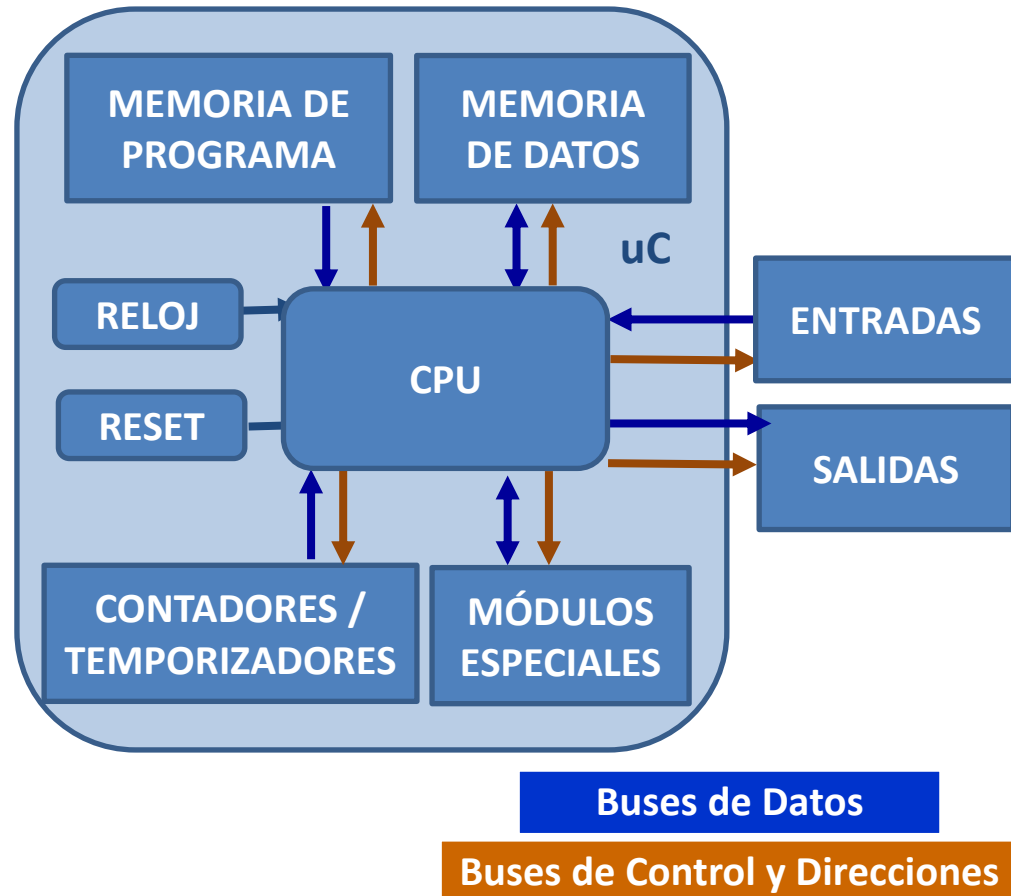
- El procesador
- Memoria de programa
- Memoria de datos
- Líneas de entrada y salida (I/O)
- Recursos auxiliares

DIFERENCIAS ARQUITECTURA

ARQUITECTURA MICROPROCESADA



ARQUITECTURA MICROCONTROLADA



CARACTERÍSTICAS BÁSICAS

UTILIDAD

- Se lo emplea para controlar una única tarea y gracias a su reducido tamaño suele incorporarse en el propio dispositivo que controla, formando un SISTEMA EMBEBIDO.

MEMORIA

- Limitada, en la que se almacena un solo programa.

PUERTOS DE ENTRADA Y SALIDA

- Se conectan con sensores y actuadores al dispositivo físico que controlan.

PROPÓSITO ESPECÍFICO

- Una vez programado el microcontrolador sólo sirve para atender la tarea para la que ha sido programado.

COMPARACIÓN RÁPIDA

CARACTERÍSTICA	ARQUITECTURA MICROPROCESADA	ARQUITECTURA BASADA EN MICROCONTROLADOR
TIPO DE SISTEMA	ABIERTO	CERRADO
CAPACIDAD DE MEMORIA	VARIOS GB INCLUSO TB	ALGUNOS MB
NÚCLEOS DE PROCESAMIENTO	MULTI/ MANY-CORE	MONO-CORE
PERIFÉRICOS	FUERA DEL μP	INCLUIDOS EN EL CIRCUITO INTEGRADO DEL μC
CONSUMO DE POTENCIA	ALTO – ORDEN DE DECIVATIOS A KILOVATIOS	BAJO – ORDEN DE VATIOS A MILIVATIOS
DESEMPEÑO	ORDEN DE TERAFLIPS	ORDEN DE POCOS MEGAFLOPS
CONFIABILIDAD	BAJA	MEDIA A ALTA
APLICACIONES	SISTEMAS DE GRAN PROCESAMIENTO DE INFORMACIÓN	SISTEMAS EMBEBIDOS

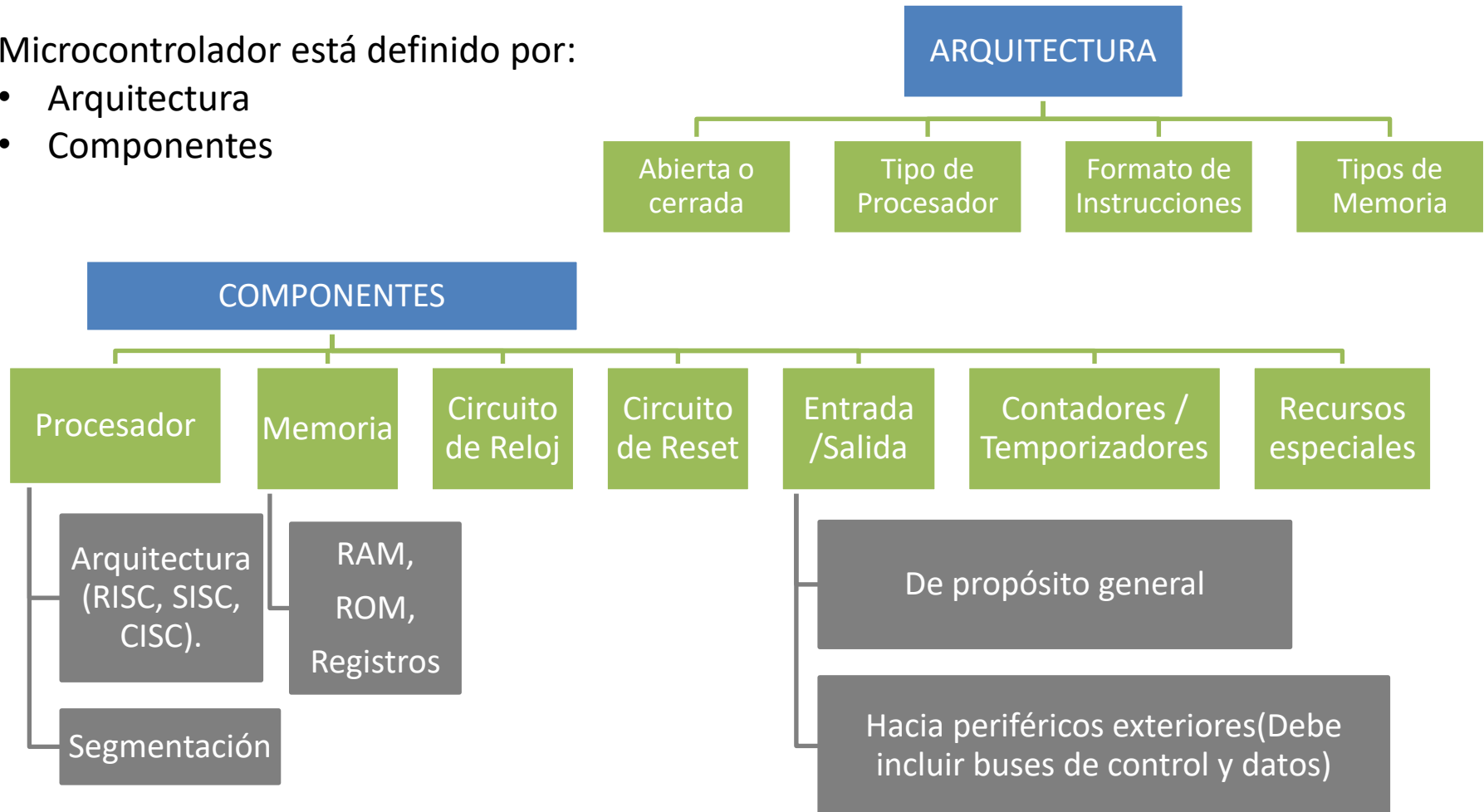
CONTENIDO

- Introducción
- **Revisión de Teoría básica de Microprocesadores**
- Procesador embebido ARM

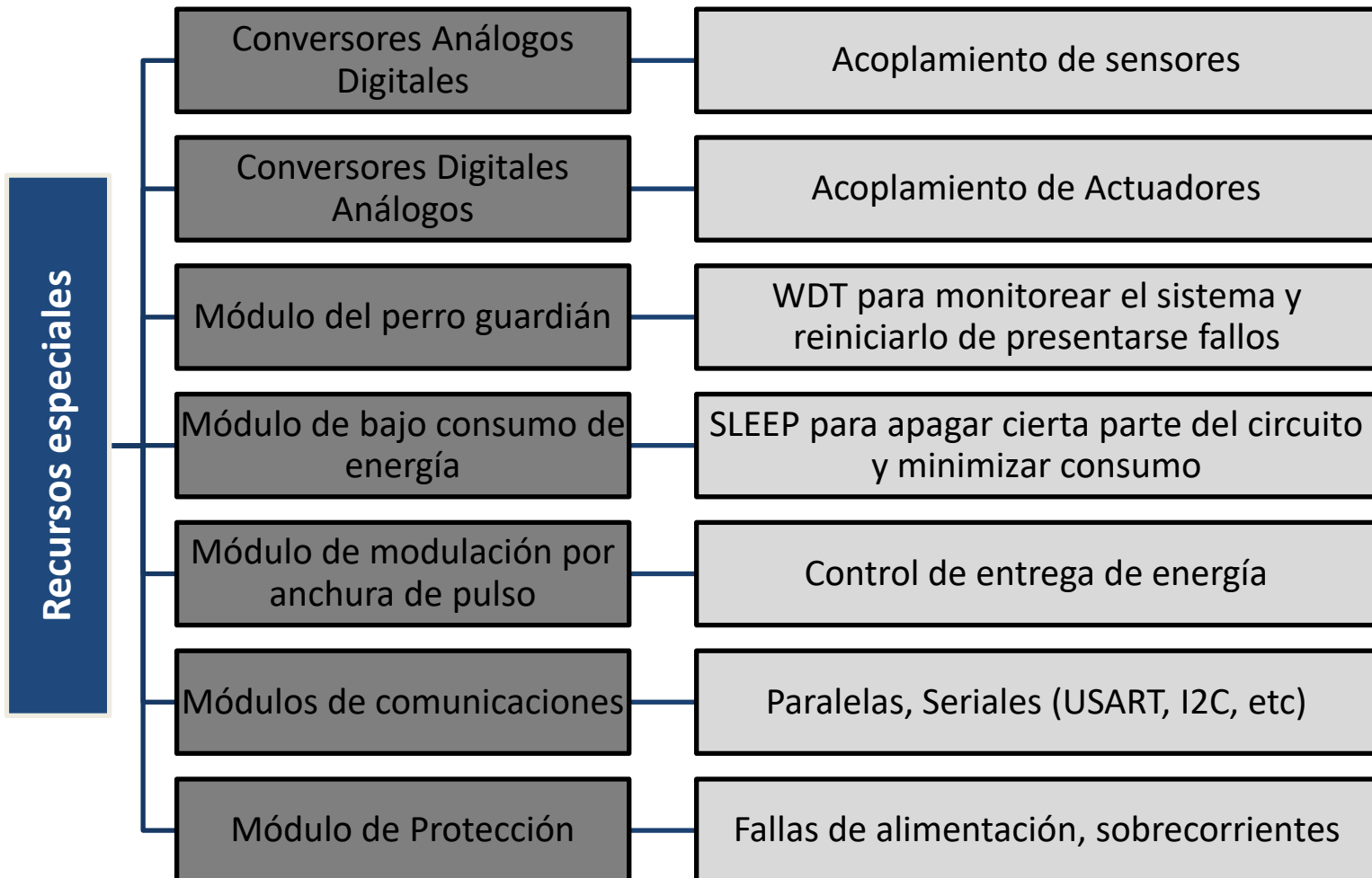
CONCEPTOS BÁSICOS

Microcontrolador está definido por:

- Arquitectura
- Componentes



RECURSOS ESPECIALES



HERRAMIENTAS DE DESARROLLO

UNA PLATAFORMA DE DESARROLLO O SISTEMA DE ENTRENAMIENTO PUEDE INCLUIR

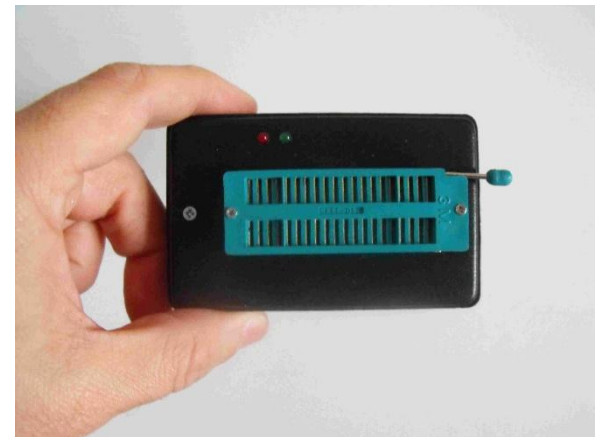
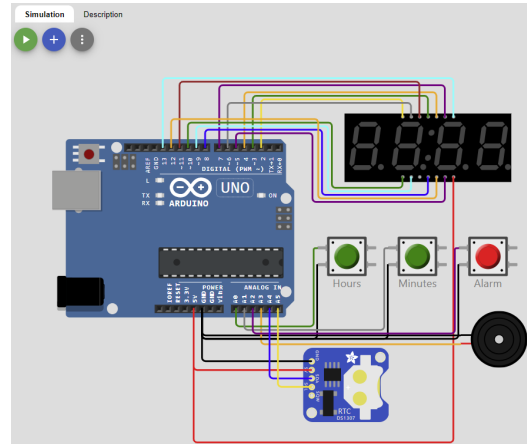
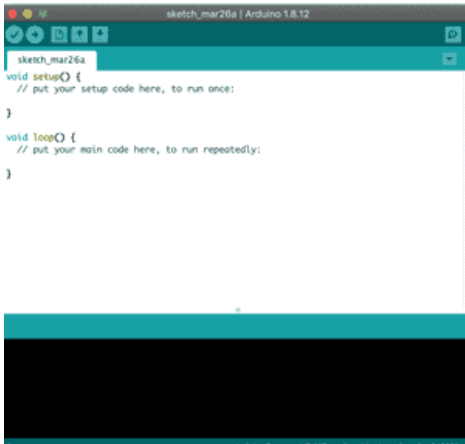
ENSAMBLADOR

COMPILADOR

DEPURADOR

SIMULADOR

PROGRAMADOR

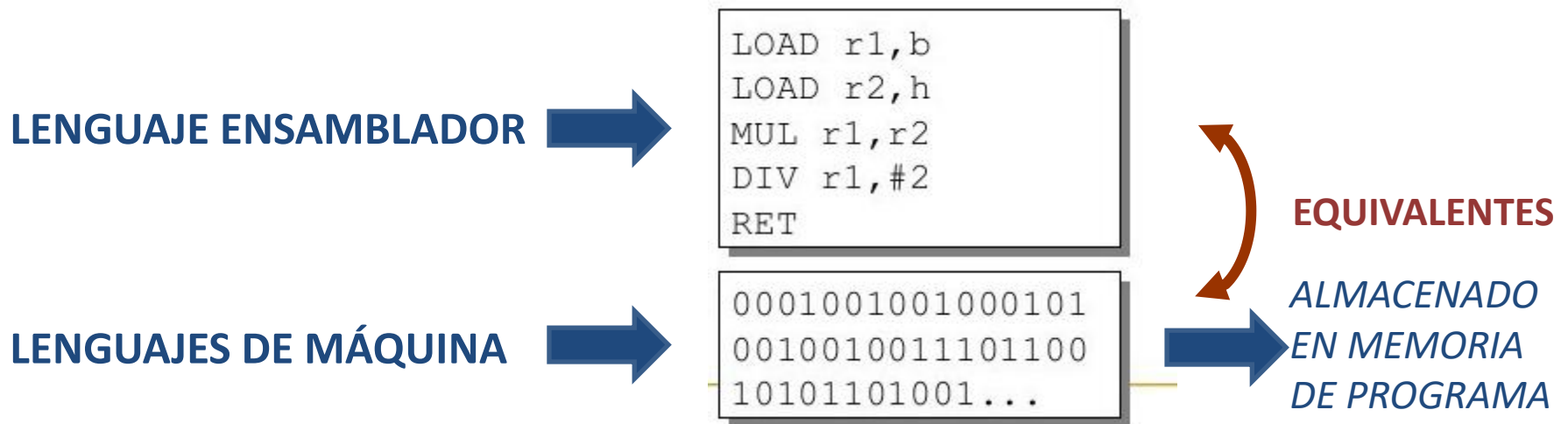


ENSAMBLADOR

Convierte código escrito en lenguaje ensamblador (nemónicos) a código binario (lenguaje de máquina)

El *lenguaje de máquina* suele ser el primer nivel al que tiene acceso el usuario de un computador, salvo si la máquina es micro-programable, en cuyo caso dispone de un nivel intermedio que permite modificar el repertorio de instrucciones.

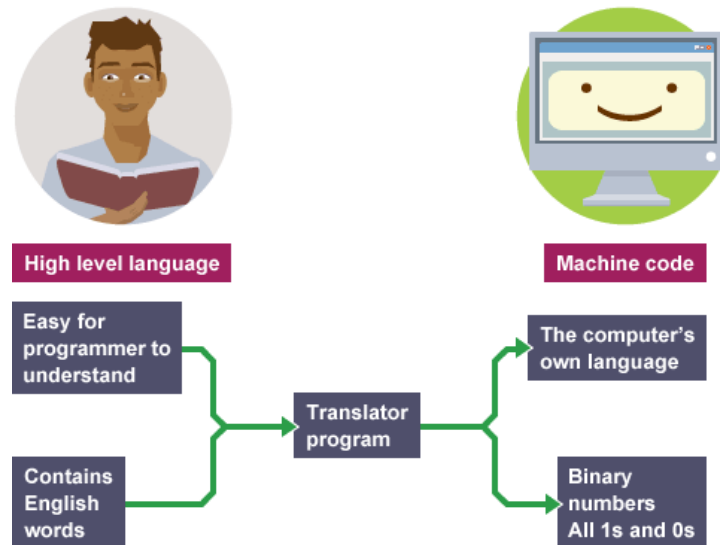
Normalmente, el usuario no utiliza directamente el lenguaje máquina, sino una representación simbólica del mismo conocida como *lenguaje ensamblador*.



COMPILADOR

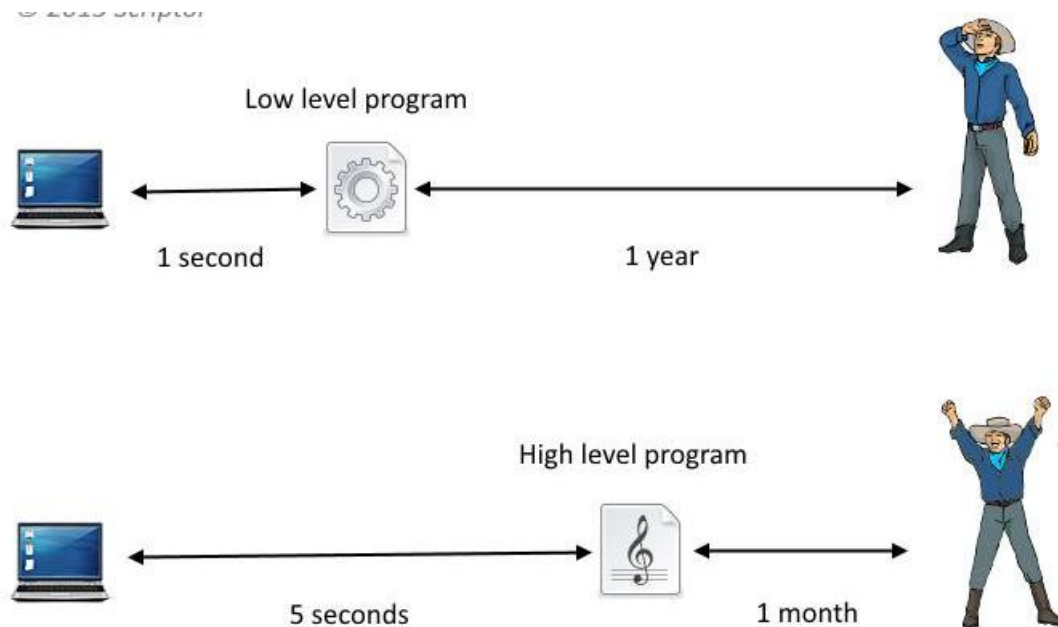
Convierte código escrito en un lenguaje de “alto nivel”, tal como Basic, C, Java, etc. a código binario ejecutable. A veces pasando por un código intermedio relocizable llamado código objeto.

El código objeto es una versión de código de máquina con referencias simbólicas a otros archivos fuentes que son resueltas después por un encadenador (linker) para convertirlo en código ejecutable.

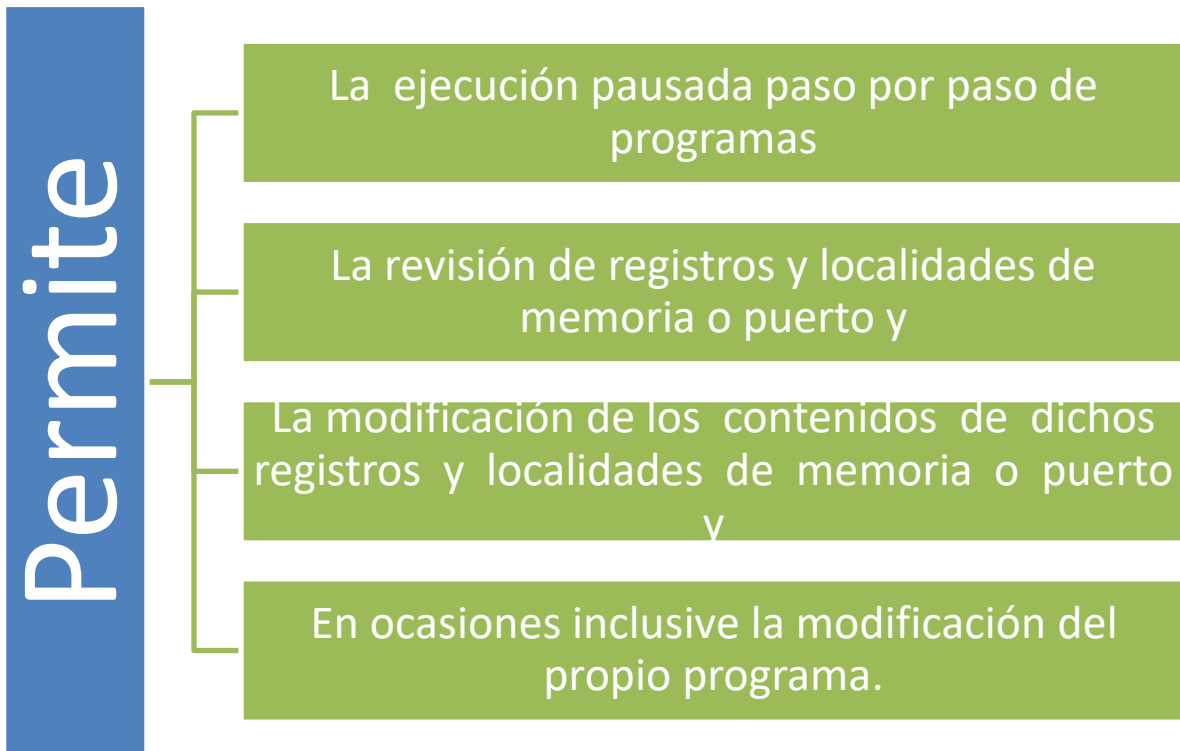


COMPILADOR

La programación en un lenguaje de alto nivel (como el C) disminuye el tiempo de desarrollo de un producto, sin embargo, el código resultante es más ineficiente que el programado en ensamblador.



DEPURADOR



SIMULADOR

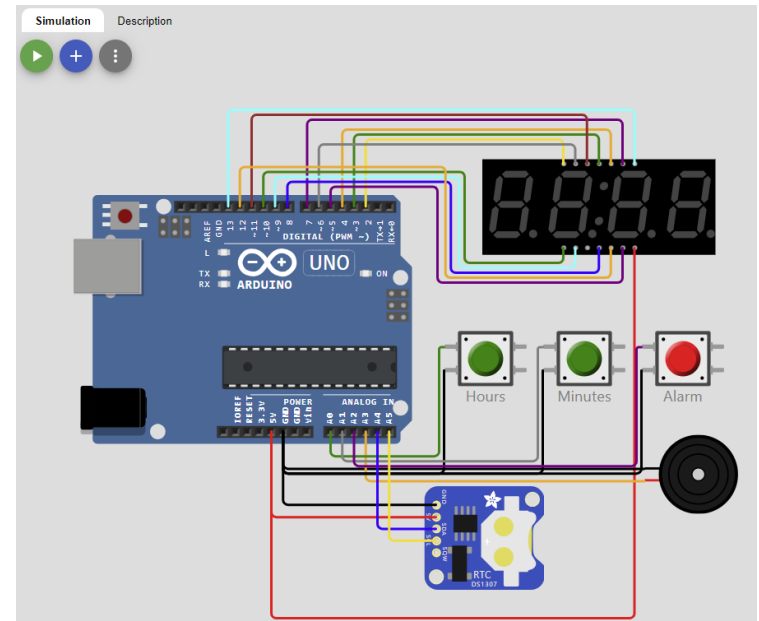
Permite la ejecución y depuración de programas de manera simulada.

La simulación se realiza sobre una CPU diferente

- No se realiza en tiempo real
- No puede simular todas las capacidades de un microcontrolador

Sin embargo, resultan ideales para una depuración por bajo costo.

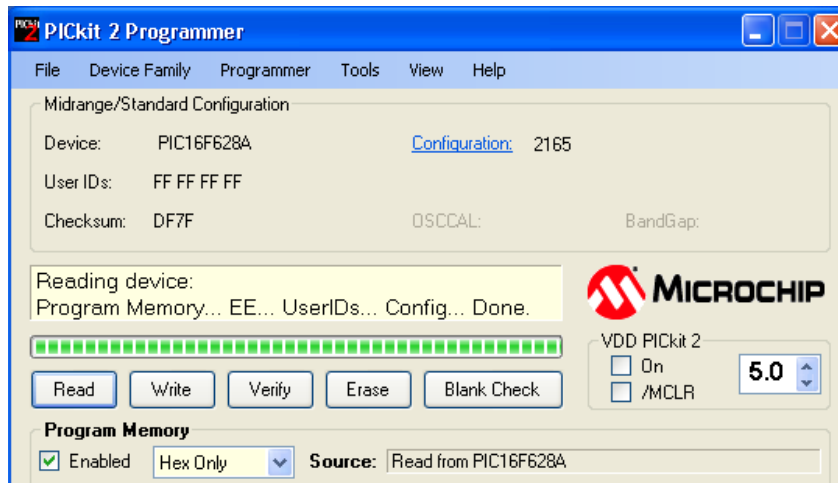
Si bien es una ayuda, debe considerarse que en condiciones reales el sistema puede requerir cambios o adecuaciones para su correcto desempeño.



PROGRAMADOR

Permite grabar información en la memoria PROM (EPROM, EEPROM o FLASH) incluida en el chip del microcontrolador.

Normalmente la información guardada en este tipo de memoria corresponde a código de programa



APLICACIONES

Instrumentos portátiles compactos:

- Radio paginador numérico (beeper)
- Nivelímetro digital

Dispositivos periféricos:

- Teclado
- Modems
- Plotters

Dispositivos autónomos (stand alone):

- Fotocopiadoras
- Codificador- decodificador de T.V.

APLICACIONES

Subfunciones de instrumentos:

- Panel frontal de un osciloscopio
- Control de pantalla sensible al tacto

Aplicaciones automotrices:

- Control de encendido e inyección de combustible
- Sistema de frenos antiderrapes (ABS)

Otros:

- Electrodomésticos
- Teléfonos celulares
- Sistemas de seguridad

- Introducción
- Revisión de Teoría básica de Microprocesadores
- **Procesador embebido ARM**

PROCESADOR EMBEBIDO ARM

Microcontroladores ARM (Advanced RISC Machine)

- La arquitectura ARM se diseñó para permitir implementaciones de tamaño muy reducido y de alto rendimiento.
- Estas arquitecturas tan simples permiten dispositivos con muy bajo consumo de energía.
- Se caracteriza fundamentalmente por ser una computadora de set de instrucciones reducido (Reduced Instruction Set Computer, RISC)
 - ARM1: primer procesador creado por Acorn Computer en 1985. Apuntaba al mercado desktop (dominado por Intel y Motorola).
 - Evoluciona en ARM2 y ARM3. Los únicos nichos eran educación y hobbists.
 - Aparecen empresas interesada en integrar procesadores ARM en sus propios diseños de sistemas embebidos. En especial Apple.
 - Nace Advanced RISC Machine (ARM), producto del aporte de capital de Apple y know how e ingeniería de Acorn.

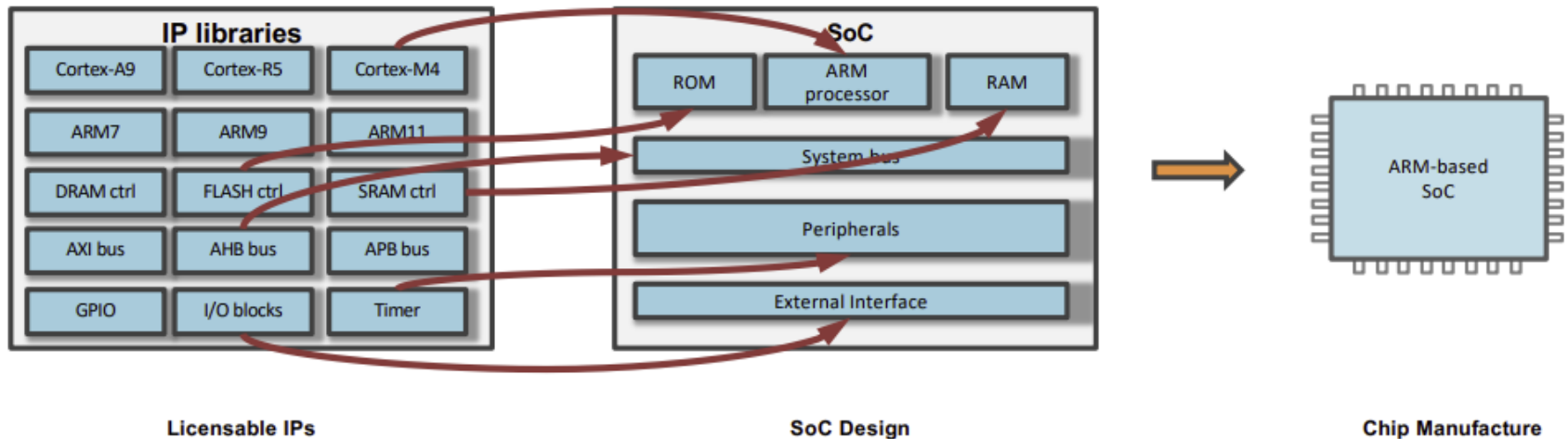
PROCESADOR EMBEBIDO ARM

Su modelo de negocios será diferente del clásico. En lugar de vender el chip con el procesador, vende los derechos de propiedad intelectual para que otras compañías empleen procesadores ARM en sus diseños embebidos .

El diseño de un Sistema en un Chip (SoC) requiere de los siguientes pasos:

- Seleccionar un conjunto de núcleos IP de ARM y/u otros IP
- Integrar los núcleos IP en el diseño de un chip
- Pedir a un fabricante de semiconductores que nos haga el chip

De esta forma el tiempo de puesta en mercado se reduce frente a un ASIC.



PROCESADOR EMBEBIDO ARM

FAMILIA DE PROCESADORES ARM

Cortex-A series (Application)

- Procesadores de altas prestaciones, capaces de ejecutar un Sistema Operativo completo como por ejemplo Linux
- Entre sus aplicaciones están: smartphones, TV, libros digitales, etc.

Cortex-R series (Real-time)

- Altas prestaciones en aplicaciones de tiempo real
- Gran fiabilidad
- Entre sus aplicaciones están: sistemas de frenado para automóviles, sistemas de seguridad etc.

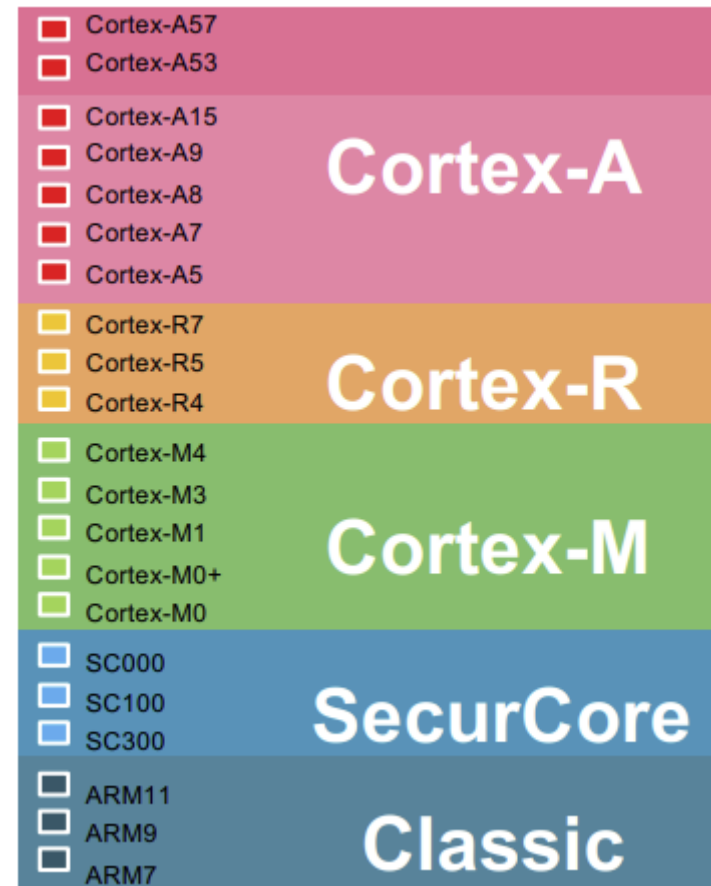
Cortex-M series (Microcontroller)

- Aplicaciones de tiempo determinista donde el precio es un factor importante
- Entre sus aplicaciones tenemos: dispositivos de señal mixta (que incluyen parte analógica y digital), sensores inteligentes, sistemas empotrados en vehículos y transportes.

SecurCore series

- Aplicaciones de alta seguridad (security, not safety).

Nomenclatura de modelos anteriores. – Familias ARM7, ARM9, ARM11



PROCESADOR EMBEBIDO ARM

Familia de procesadores ARM Cortex-M

Processor	ARM Architecture	Core Architecture	Thumb*	Thumb*-2	Hardware Multiply	Hardware Divide	Saturated Math	DSP Extensions	Floating Point
Cortex-M0	ARMv6-M	Von Neumann	Most	Subset	1 or 32 cycle	No	No	Software	No
Cortex-M0+	ARMv6-M	Von Neumann	Most	Subset	1 or 32 cycle	No	No	Software	No
Cortex-M1	ARMv6-M	Von Neumann	Most	Subset	3 or 33 cycle	No	No	Software	No
Cortex-M3	ARMv7-M	Harvard	Entire	Entire	1 cycle	Yes	Yes	Software	No
Cortex-M4	ARMv7E-M	Harvard	Entire	Entire	1 cycle	Yes	Yes	Hardware	Optional

PROCESADOR EMBEBIDO ARM

ARQUITECTURA DEL PROCESADOR CORTEX-M0+

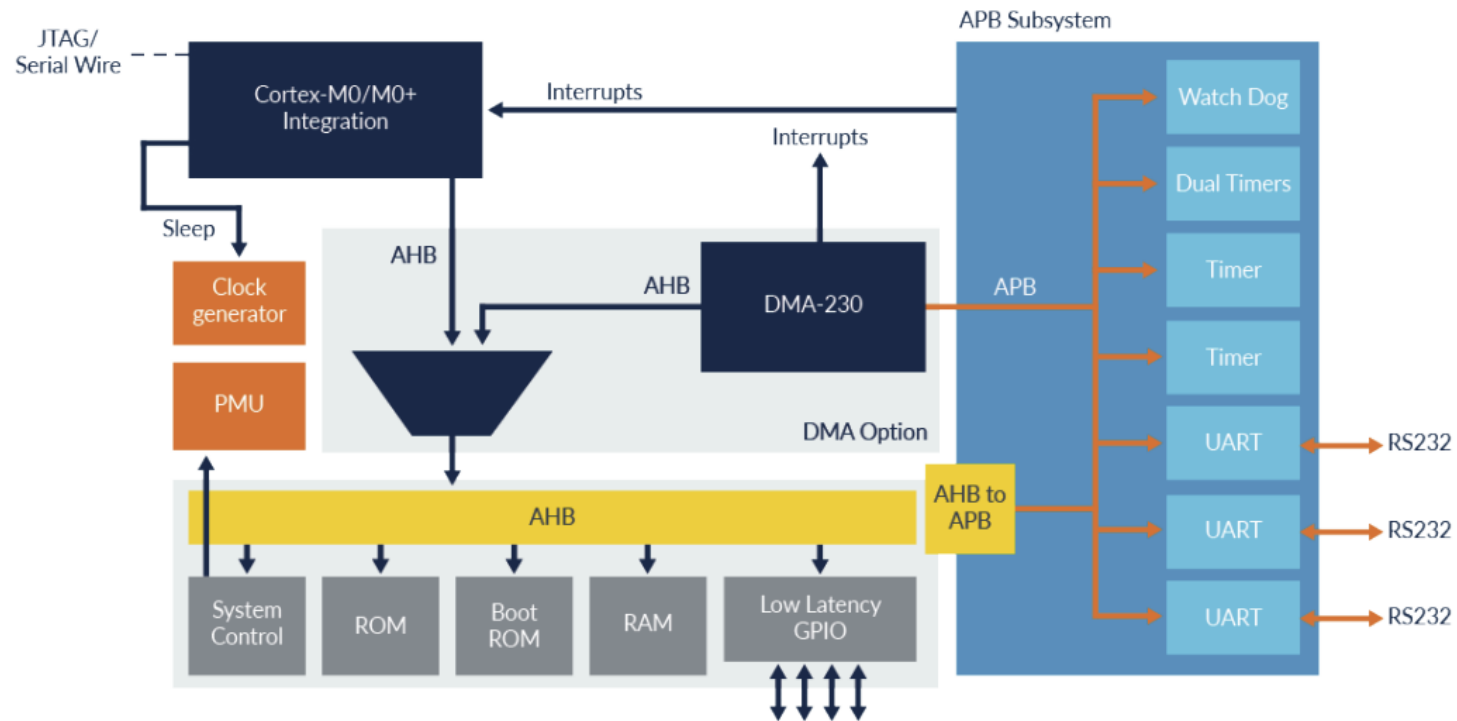
El ARM C rtex ARM M0+ es un microcontrolador RISC con arquitectura Von Neumann, pues consta de un solo bus para datos e instrucciones. Este microcontrolador utiliza un conjunto de instrucciones llamado THUMB, dicho juego de instrucciones es en su mayor a de 16 bits, aunque existen instrucciones de 32 bits para ciertas acciones.

El ARM C rtex M0+ tiene:

- Un conjunto de 56 instrucciones base
- El ARM C rtex M0+ consta de 2 niveles de pipeline
- 16 registros de 32 bits
- NVIC (Nested Vectored interrupt Controller) y
- 1 multiplicador de 32 bits que var a seg n el fabricante

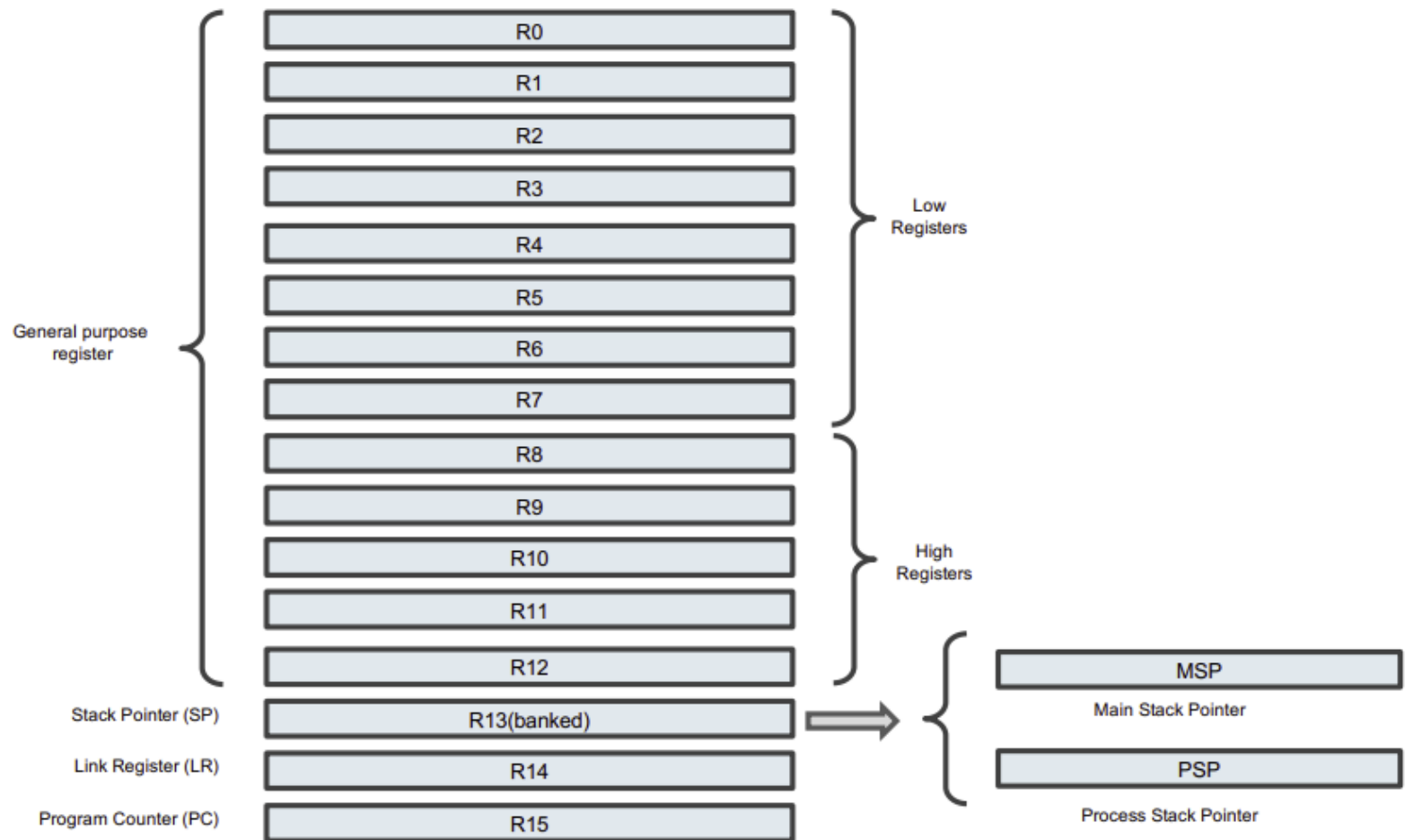
ARQUITECTURA INTERNA Y DEL SET DE INSTRUCCIONES (ISA) ARM

EJEMPLO DE ARQUITECTURA DEL PROCESADOR CORTEX-M0+



ARQUITECTURA INTERNA Y DEL SET DE INSTRUCCIONES (ISA) ARM

REGISTROS DEL PROCESADOR



ARQUITECTURA INTERNA Y DEL SET DE INSTRUCCIONES (ISA) ARM

Descripción de los registros del procesador

R0 – R12: Registros de propósito general

- Los registros (R0 – R7) se pueden utilizar en cualquier instrucción
- Los registros (R8 – R12) no pueden utilizarse siempre.

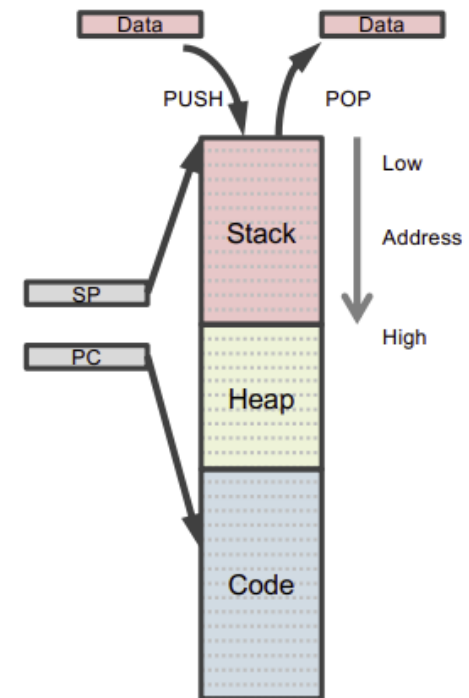
Por ejemplo, no son accesibles para algunas instrucciones Thumb (16-bit)

R13: Puntero de Pila (SP)

- Guarda el valor actual de la cima de pila
- Se usa para guardar el contexto de un programa cuando se cambia a otra tarea
- Algunos procesadores tiene dos SPs:
 - + el SP principal, usado en aplicaciones que requieren acceso privilegiado, como por ejemplo en el kernel de un sistema operativo, o en el manejo de excepciones,
 - + el SP de proceso, usado en aplicaciones básicas.

Contador de Programa (PC)

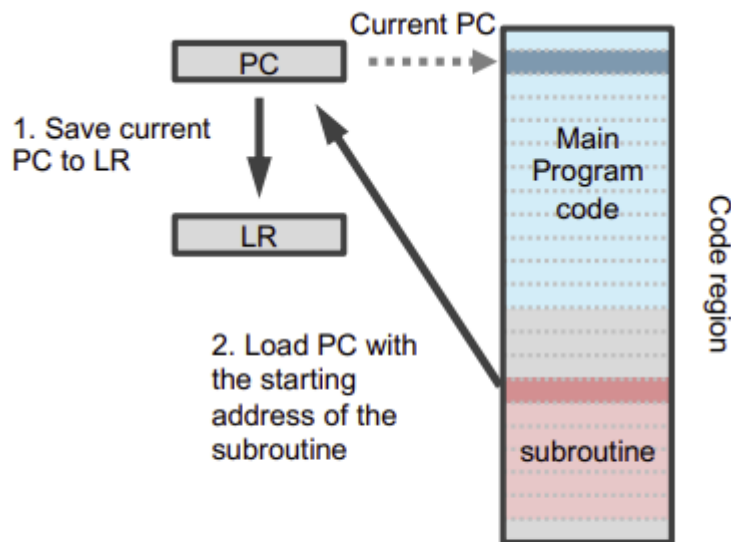
- Contiene la dirección de la instrucción actual
- Automáticamente se incrementa en 4 al finalizar cada instrucción (para instrucciones de 32-bit), excepto si la instrucción provoca un salto
- Una instrucción de salto, tal como la llamada a una función, cambia expresamente el PC a una dirección específica, y guarda el valor del PC (dirección de retorno) al Link Register (LR)



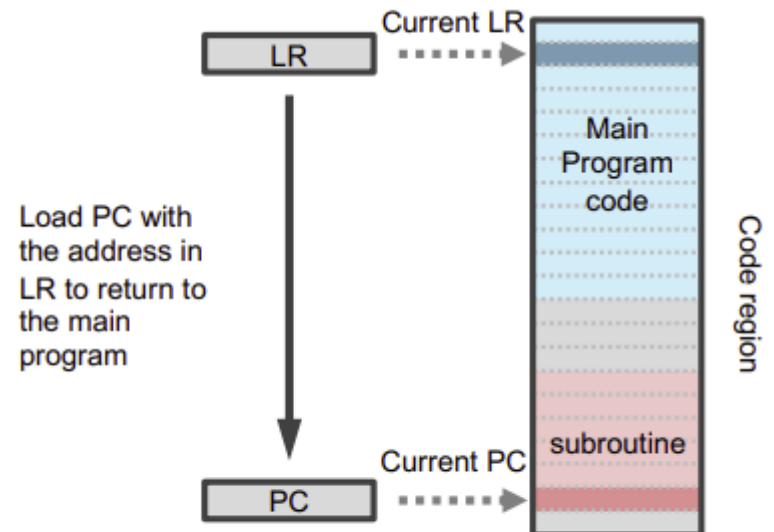
Descripción de los registros del procesador

R14: Link Register (LR)

- LR se usa para almacenar la dirección de retorno de una subrutina o una llamada a una función
- El contador de programa (PC) se cargará con el valor de LR una vez la función finaliza



Call a subroutine



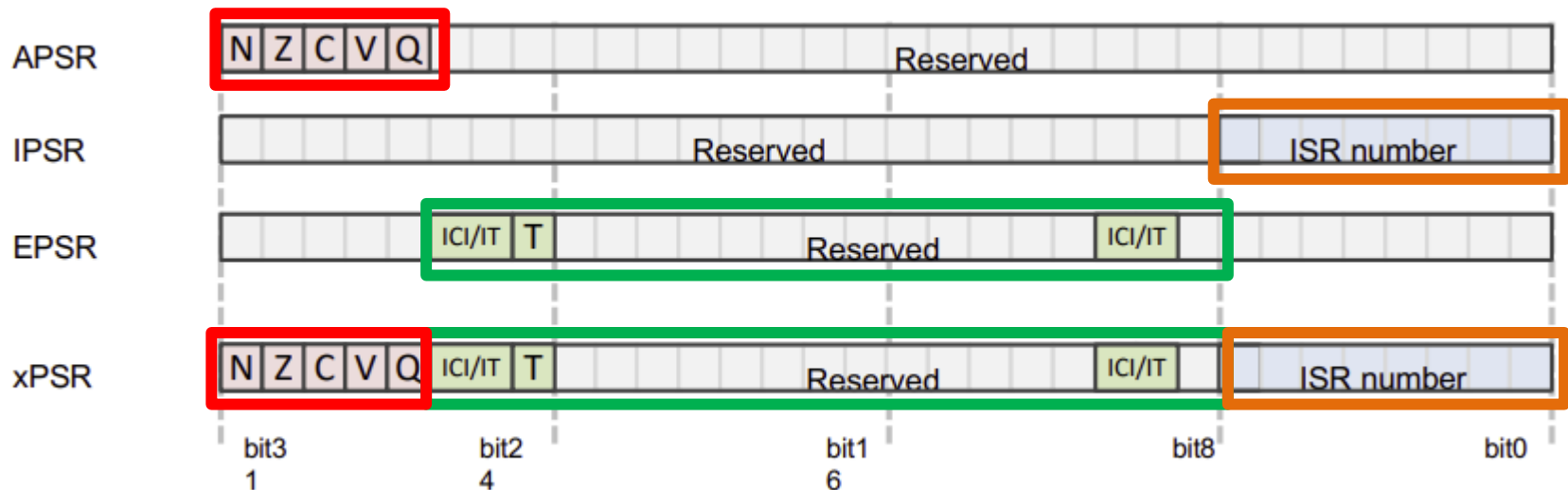
Return from a subroutine to the main program

ARQUITECTURA INTERNA Y DEL SET DE INSTRUCCIONES (ISA) ARM

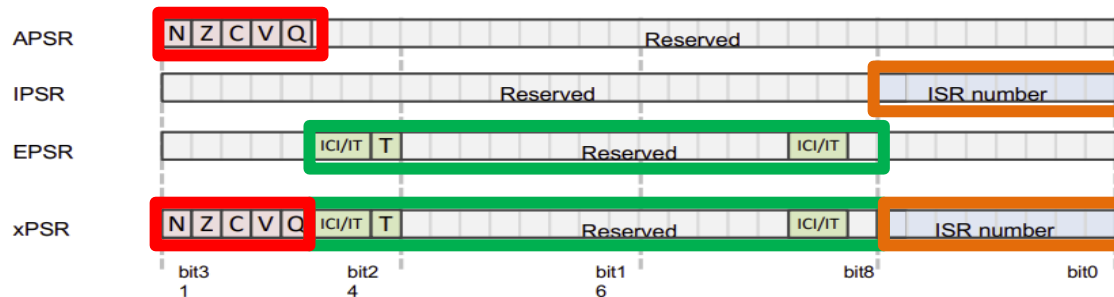
Descripción de los registros del procesador

xPSR, es el registro de estado (Program Status Register)

- Nos proporciona información sobre la ejecución del programa y sobre las banderas de la ALU
- Lo podemos dividir en tres zonas
 - Application PSR (APSR)
 - Interrupt PSR (IPSR)
 - Execution PSR (EPSR)



Descripción de los registros del procesador

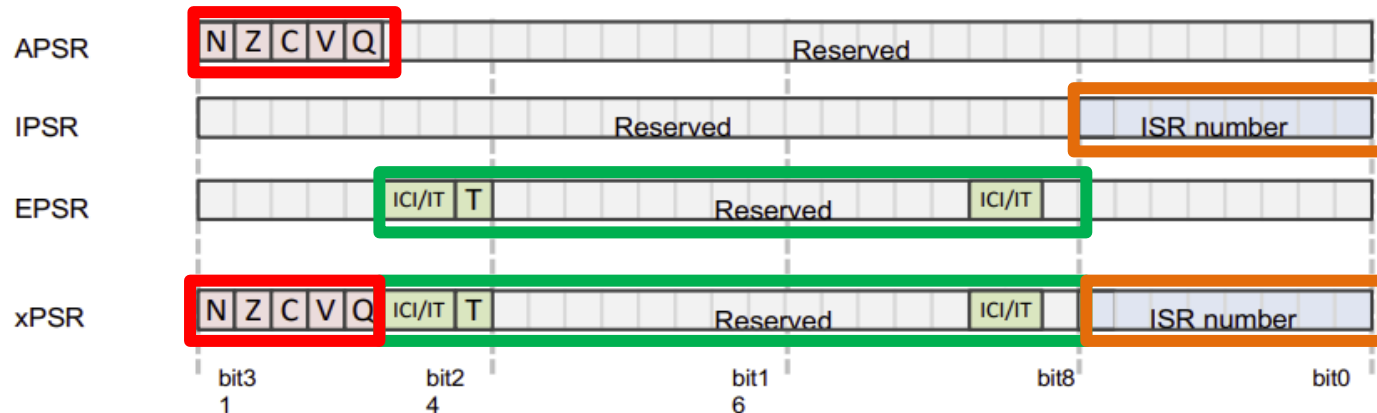


• APSR

- N: negative flag – indicador de resultado negativo: vale “1” si el resultado de la ALU es negativo
- Z: zero flag – indicador de cero: se pone a “1” si el resultado de la ALU es cero
- C: carry flag – indicador de acarreo: se pone a “1” si se produce desbordamiento en una operación sin signo
- V: overflow flag – indicador de desbordamiento: se pone a “1” si se produce desbordamiento en una operación con signo
- Q: sticky saturation flag – indicador de saturación: se pone a “1” si se produce saturación en una instrucción de aritmética saturada, o desbordamiento en ciertas instrucciones de multiplicación

ARQUITECTURA INTERNA Y DEL SET DE INSTRUCCIONES (ISA) ARM

Descripción de los registros del procesador



- APSR

- IPSR

- ISR number – número de subrutina de interrupción que se está ejecutando en un momento dado

- EPSR

- T: Thumb state – siempre a “1” ya que Cortex-M4 solo funciona en estado “Thumb”
 - IC/IT: Interrupt-Continuable Instruction (ICI) bit, IF-THEN instruction status bit

SET DE INSTRUCCIONES (ISA) ARM

Conjunto de instrucciones ARM inicial

- Conjunto de instrucciones de 32 bits, llamado instrucciones ARM.
- Potente y buen rendimiento.
- Memoria de programa más grande en comparación con los procesadores de 8 y 16 bits.
- Mayor consumo de energía

Conjunto de instrucciones Thumb-1

- Conjunto de instrucciones de 16 bits, utilizado por primera vez en el procesador ARM7TDMI en 1995
- Proporciona un subconjunto de las instrucciones ARM, lo que proporciona una mejor densidad de código en comparación con la arquitectura RISC de 32 bits
- El tamaño del código se reduce en un $\sim 30\%$, pero el rendimiento también se reduce en un $\sim 20\%$

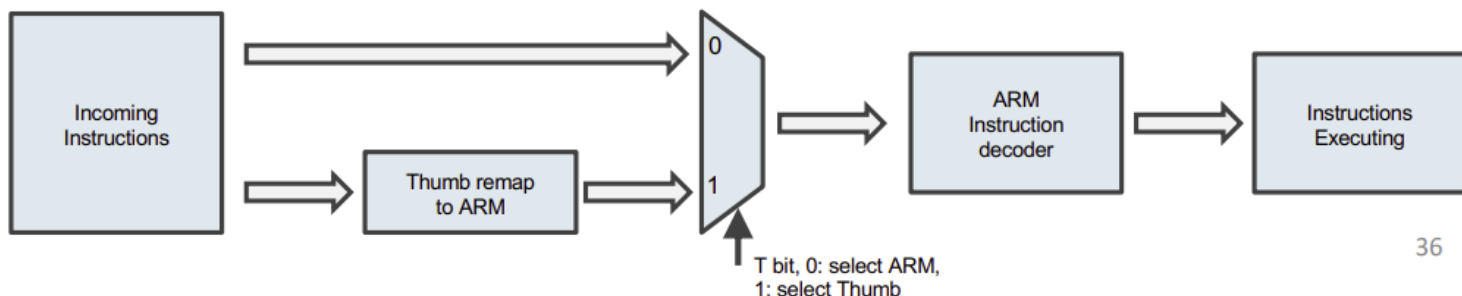
SET DE INSTRUCCIONES (ISA) ARM

Mezcla de conjuntos de instrucciones ARM y Thumb-1

- Se juntan los beneficios de las instrucciones ARM de 32 bits (alto rendimiento) y Thumb-1 de 16 bits (alta densidad de código)
- Se utiliza un multiplexor para cambiar entre dos estados: el “estado ARM” (32 bits) y el “estado Thumb” (16 bits)

Conjunto de instrucciones Thumb-2

- Consta de instrucciones Thumb de 32 bits y conjuntos de instrucciones Thumb-1 originales de 16 bits.
- En comparación con el conjunto de instrucciones ARM de 32 bits, el tamaño del código se reduce en aproximadamente un 26%, mientras se mantiene un rendimiento similar
- Capaz de manejar los requisitos de procesamiento en un único estado



ARQUITECTURA INTERNA Y DEL SET DE INSTRUCCIONES (ISA) ARM

SET DE INSTRUCCIONES (ISA) ARM

Sintaxis del código ensamblador ARM:

Label mnemonic operando1, operando2, ... ; Comentarios

- **Label** se usa como referencia de una dirección;
- **Mnemonic** es el nombre de la instrucción;
- **Operando1** es el destino de la operación;
- **Operando2** es usualmente el origen de la operación;
- **Los comentarios se escriben después de un “ ; ”, y no afectan al programa;**

Por ejemplo

MOVS R3, #0x11 ;Guardar 0x11 en el registro R3

El código ensamblador puede compilarse con ARM assembler (armasm) o herramientas de ensamblador de las disponibles en el mercado (e.g. GNU tool chain). Cuando se usan estas otras, la sintaxis de las etiquetas (labels) y comentarios puede ser ligeramente diferente.